开发入门指南

修订页

版本号	修订（内容）说明	修订时间
V1.0.0	C++ 版本上线，新建文档（对应 SDK 版本号 V3.0.0）	2026/04/30

---

1. 兼容性与要求

在开始使用灵巧手 SDK 之前，请确认开发环境满足以下要求。

1.1 硬件要求

项目	要求
CPU 架构	x86 / x86_64
内存	建议 4 GB 及以上
通讯接口	USB、RS-485（USB 转 485 转换器）、以太网（EtherCAT）

1.2 操作系统要求

项目	要求
操作系统	Ubuntu 22.04 LTS
内核版本	5.15 及以上（Ubuntu 22.04 默认内核）

1.3 通讯协议支持

协议	接口	典型用途
Modbus-RTU	RS-485 / USB 转 485	学术研究、小规模部署
USB	USB-A / USB-C	直连单设备调试

---

2. 安装指南

本节介绍 GMH15 灵巧手的安装方法以及常见安装问题的解决方案。

2.1 运行环境要求

项目	要求
操作系统	Ubuntu 22.04 LTS
CPU	64 bit 处理器
内存	4 GB RAM 及以上
存储	20 GB 可用空间及以上
C++	C++ 17 标准
其他依赖库	CMake >= 3.22；Boost 1.74.0；OpenCV 4.5.4；spdlog 1.9.2；fmt 8.1.1

2.2 安装 DexHandSDK

安装 DexHandSDK 依赖库：

```
# 依赖项（如无法一次性进行安装，可逐个安装依赖）
sudo apt update && sudo apt install -y \
    build-essential \
    libboost-all-dev \
    libopencv-dev \
    libspdlog-dev \
    libfmt-dev \
    libgoogle-glog-dev \
    libopenblas-dev \
    libdlib-dev \
    libjsoncpp-dev \
    libfltk1.3-dev \
    libyaml-cpp-dev \
    python3-pip \
    liburdfdom-dev \
    liburdfdom-headers-dev \
    python3-dev \
    cmake \
    vim \
    git

# 源码编译安装 gsl
tar -xzf gsl.tar.gz
cd gsl
./configure CFLAGS="-g -O0" --prefix="/usr"
make -j4
sudo make install

# 源码编译安装 eigen3
tar -xzf eigen-3.3.7.tar.gz
cd eigen-3.3.7
mkdir build
cd build
```

```
cmake ..
sudo make install
```

安装 DexHandSDK：

```
# Linux C++ 版本安装
sudo dpkg -i DexHandSDK-3.0.0-Linux.deb
```

2.3 故障排除

问题一：串口权限不足

现象：连接时提示 `Permission denied: '/dev/ttyUSB0'`

解决方案：

```
# 通过插拔的方式确认设备是否存在
ls /dev/ttyUSB*

# 临时授权查询到的设备（重启后失效），以 /dev/ttyUSB0 为例
sudo chmod 666 /dev/ttyUSB0

# 永久授权（推荐）
sudo usermod -aG dialout $USER
# 注销后重新登录生效
```

问题二：找不到 USB 设备

现象：`ls /dev/ttyUSB*` 无输出

解决方案：

```
# 检查 USB 设备是否被系统识别
lsusb

# 查看内核日志确认连接状态
dmesg | tail -20

# 加载 USB 串口驱动（若未自动加载）
sudo modprobe cp210x   # Silicon Labs 芯片
sudo modprobe ch341    # CH340/CH341 芯片
```

问题三：`scanModbusDevices()` 扫描超时或返回空列表

原因及处理：

可能原因	处理方式
波特率不匹配	改用 `connectModbusAuto()` 自动尝试波特率
RS-485 接线错误（A/B 反接）	检查 A+/B- 接线极性
设备未上电	检查灵巧手电源状态
从站 ID 冲突	确认总线上各设备 ID 唯一

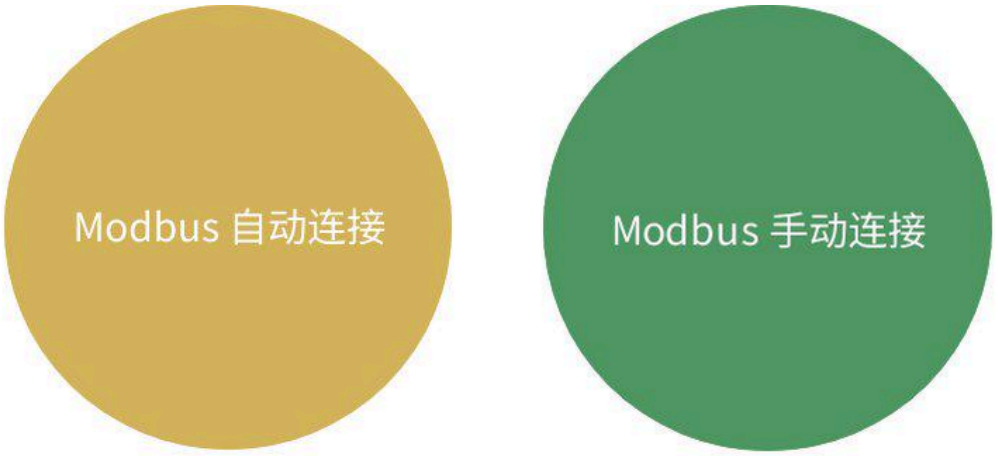
---

3. 快速上手

3.1 设备连接

目前，DexHandSDK 支持 Modbus 通讯协议和 USB 2.0 通讯协议。

对于 Modbus 通讯协议，DexHandSDK 提供了两种连接方式，如下所示。



方式	适用场景
Modbus 自动连接	通过 USB 转 485 转换器连接，无需手动配置波特率和从站地址
Modbus 手动连接	已知波特率和从站 ID，需要精确控制连接参数

DexHandSDK Modbus 自动连接方式的使用步骤：



示例代码：

```
#include <dexh15/dexh15_control.h>
```

```cpp
#include <iostream>

/**
 * @brief Modbus 自动连接
 * @details 步骤:
 *          (1) 自动连接 Modbus 串口下的第一个从站
 *          (2) 查询灵巧手在线状态
 *          (3) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    // Modbus 串口号
    std::string hand_port_num = "/dev/ttyUSB0";
    // Modbus 从站地址
    int16_t device_address;
    // 自动连接 Modbus 串口下的第一个从站
    if (!dexh15->connectModbusAuto(hand_port_num, device_address))
    {
        std::cerr << "请检查灵巧手是否接入 Modbus 串口设备" << std::endl;
        return 1;
    }
    std::cout << "自动连接灵巧手成功" << std::endl;
    // 查询灵巧手在线状态
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手" << (is_connected ? "在线" : "离线") << std::endl;
    // 关闭 Modbus 串口
    bool result = dexh15->disconnectModbus();
    std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_connected ? " 成功"
: " 失败") << std::endl;
    return 0;
}
```

DexHandSDK Modbus 手动连接方式的使用步骤:



示例代码:

```cpp
#include <dexh15/dexh15_control.h>

#include <iostream>

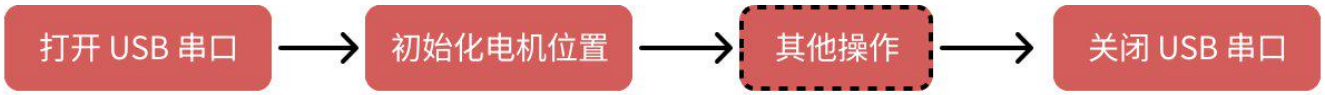
/**
 * @brief Modbus 手动链接
 * @details 步骤:
 *          (1) 打开 Modbus 串口
 *          (2) 初始化 Modbus 从站
 *          (3) 初始化电机位置（如果刚上电时已调用，可跳过）
 *          (4) 查询灵巧手在线状态
```

```
 *          (5) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    // Modbus 串口号
    std::string hand_port_num = "/dev/ttyUSB0";
    // 波特率
    int32_t baud_rate = 4000000;
    // Modbus 从站地址
    int16_t device_address = 0x79;
    // 打开 Modbus 串口
    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
    {
        std::cerr << "请检查是否插入 Modbus 串口设备" << std::endl;
        return 1;
    }
    std::cout << "打开 Modbus 串口成功" << std::endl;
    // 初始化 Modbus 从站
    if (!dexh15->initModbusDevice(device_address))
    {
        std::cerr << "请检查灵巧手是否接入 Modbus 串口设备" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "初始化 Modbus 从站成功" << std::endl;
    // 初始化电机位置（如果刚上电时已调用，可跳过）
    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "初始化电机位置失败" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "初始化电机位置成功" << std::endl;
    // 查询灵巧手在线状态
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手" << (is_connected ? "在线" : "离线") << std::endl;
    // 关闭 Modbus 串口
    bool result = dexh15->disconnectModbus();
    std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_connected ? " 成功"
: " 失败") << std::endl;
    return 0;
}
```

对于 USB 2.0 通讯协议，DexHandSDK 提供了一种连接方式：

> **场景**：单手调试，或学术研究等场景

DexHandSDK USB 直连方式的使用步骤：



示例代码:

```
#include <dexh15/dexh15_control.h>

#include <iostream>

/**
 * @brief USB 直连
 * @details 步骤:
 *          (1) 打开 USB 串口
 *          (2) 初始化电机位置（如果刚上电时已调用，可跳过）
 *          (3) 查询灵巧手在线状态
 *          (4) 关闭 USB 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    // USB 串口号
    std::string hand_port_num = "/dev/ttyACM0";
    // 从站地址
    int16_t device_address = 0x79;
    // 打开 USB 串口
    if (!dexh15->openUsbDevice(hand_port_num, device_address))
    {
        std::cerr << "请检查是否插入 USB 串口设备" << std::endl;
        return 1;
    }
    std::cout << "打开 USB 串口成功" << std::endl;
    // 初始化电机位置（如果刚上电时已调用，可跳过）
    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "初始化电机位置失败" << std::endl;
        dexh15->disconnectUsb();
        return 1;
    }
    std::cout << "初始化电机位置成功" << std::endl;
    // 查询灵巧手在线状态
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手" << (is_connected ? "在线" : "离线") << std::endl;
    // 关闭 USB 串口
    bool result = dexh15->disconnectUsb();
    std::cout << "断开 USB 串口设备 " << hand_port_num << (is_connected ? " 成功" : "
失败") << std::endl;
    return 0;
}
```

3.2 典型使用流程

本节提供了 6 个示例：位置模式、故障诊断、传感器数据、预定义动作、力位混合控制模式、相机功能。



3.2.1 位置模式

DexHandSDK 位置模式的使用步骤：



示例代码：

```
#include <dexh15/dexh15_control.h>
#include <dexh15/dexh15_define.h>

#include <iostream>
#include <cmath>
#include <sstream>
#include <thread>

/**
 * @brief 位置模式
 * @details 步骤:
 *          (1) 打开 Modbus 串口
 *          (2) 初始化 Modbus 从站
 *          (3) 初始化电机位置（如果刚上电时已调用，可跳过）
 *          (4) 查询灵巧手在线状态
 *          (5) 设置位置模式
 *          (6) 使能电机
 *          (7) 查询使能状态
 *          (8) 设置电机位置
```

```
 *          (9) 电机回零
 *          (10) 下使能电机
 *          (11) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate)) {
        std::cerr << "请检查是否插入 Modbus 串口设备" << std::endl;
        return 1;
    }
    if (!dexh15->initModbusDevice(device_address)) {
        std::cerr << "请检查灵巧手是否接入 Modbus 串口设备" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    if (!dexh15->initMotorPosition(device_address)) {
        std::cerr << "初始化电机位置失败" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手连接状态为" << (is_connected ? "在线" : "离线") << std::endl;

    // 设置位置模式
    if (!dexh15->setMotorControlMode(device_address,
                                     paxini::bot::dexh15::POSITION_CONTROL_MODE)) {
        std::cerr << "设置位置模式失败，无法进行位置模式控制" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    // 使能电机
    if (!dexh15->enableMotor(device_address)) {
        std::cerr << "使能电机失败" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    if (!dexh15->isMotorEnabled(device_address)) {
        std::cerr << "电机状态为未使能" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }

    // 设置电机位置
    paxini::bot::dexh15::Dex15MotorPosition positions{8000, 8000, 8000, 8000,
                                                      3000, 5000, 5000};
    if (!dexh15->setMotorTargetPosition(device_address, positions)) {
        std::cerr << "设置电机位置失败" << std::endl;
        dexh15->disconnectModbus();
```

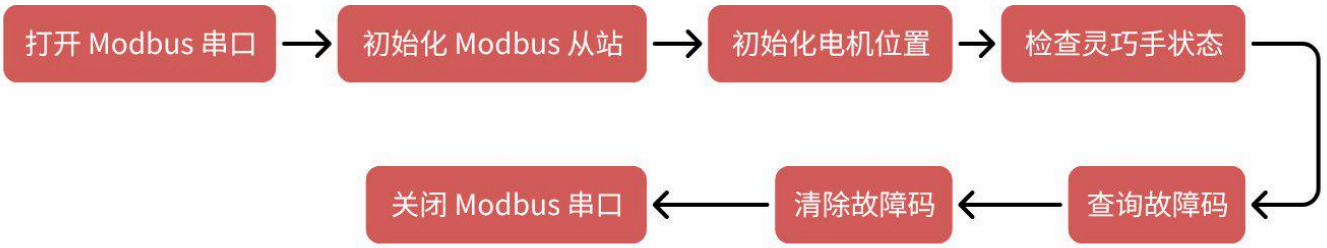
```
            return 1;
        }
        std::this_thread::sleep_for(std::chrono::seconds(1));

        // 电机回零位
        paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};
        dexh15->setMotorTargetPosition(device_address, zero_positions);

        bool is_success = dexh15->disableMotor(device_address);
        std::cout << "电机下使能" << (is_success ? "成功" : "失败") << std::endl;
        is_success = dexh15->disconnectModbus();
        std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_success ? " 成功" :
" 失败") << std::endl;
        return 0;
}
```

3.2.2 故障诊断

DexHandSDK 故障诊断的使用步骤:



示例代码:

```
#include <dexh15/dexh15_control.h>

#include <iostream>

/**
 * @brief 故障诊断
 * @details 步骤:
 *          (1) 打开 Modbus 串口
 *          (2) 初始化 Modbus 从站
 *          (3) 初始化电机位置（如果刚上电时已调用，可跳过）
 *          (4) 查询灵巧手在线状态
 *          (5) 检查灵巧手状态
 *          (6) 查询电机故障码
 *          (7) 清除电机故障码
 *          (8) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
```

```
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate)) { return 1; }
    if (!dexh15->initModbusDevice(device_address)) { dexh15->disconnectModbus();
return 1; }
    if (!dexh15->initMotorPosition(device_address)) { dexh15->disconnectModbus();
return 1; }

    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手" << (is_connected ? "在线" : "离线") << std::endl;

    // 检查灵巧手状态
    int hand_status = dexh15->checkHandStatus(device_address);
    std::cout << "灵巧手当前状态为"
              << (hand_status == 1 ? "正常运行状态"
                                   : hand_status == 2 ? "容错运行状态"
                                                      : "不可运行状态")
              << std::endl;

    // 查询电机故障码
    std::vector<paxini::bot::dexh15::FingerMotorsError> fault_codes =
        dexh15->getFaultCode(device_address);
    for (const auto &error : fault_codes) {
        std::cout << "Finger: " << error.finger_name
                  << ", Motor ID: " << error.motor_ID
                  << ", Error code: " << static_cast<int>(error.error_code)
                  << std::endl;
    }

    // 清除电机故障码
    bool result = dexh15->clearFaultCode(device_address);
    std::cout << "清除电机故障码" << (is_connected ? "成功" : "失败") << std::endl;
    result = dexh15->disconnectModbus();
    std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_connected ? " 成功"
: " 失败") << std::endl;
    return 0;
}
```

3.2.3 传感器数据

DexHandSDK 传感器数据的使用步骤:



示例代码:

```
#include <dexh15/dexh15_control.h>

#include <sstream>
```

```cpp
#include <thread>

/**
 * @brief 读取传感器数据
 * @details 步骤:
 *          (1) 打开 Modbus 串口
 *          (2) 初始化 Modbus 从站
 *          (3) 校准传感器
 *          (4) 读取传感器合力
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate)) { return 1; }
    if (!dexh15->initModbusDevice(device_address)) { dexh15->disconnectModbus();
return 1; }

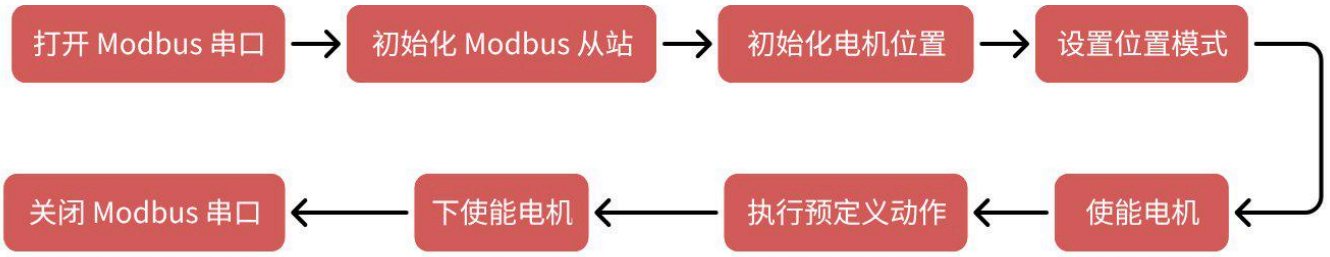
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // 校准传感器
    if (!dexh15->calibrateSensor(device_address)) {
        std::cerr << "校准传感器失败" << std::endl;
    }
    std::cout << "校准传感器成功" << std::endl;

    // 获取传感器数据（按住指尖传感器，数值才有变化）
    int hz = 10;
    while (true) {
        std::vector<paxini::bot::dexh15::ForcePoint> force_points =
            dexh15->getFingerTactile(device_address);
        for (auto force_point : force_points) {
            std::cout << "x:" << force_point.x
                      << ", y:" << force_point.y
                      << ", z:" << force_point.z << std::endl;
        }
        std::this_thread::sleep_for(
            std::chrono::milliseconds(static_cast<int>(1000.0 / hz)));
    }
    return 0;
}
```

3.2.4 预定义动作

DexHandSDK 预定义动作的使用步骤:



示例代码:

```
#include <dexh15/dexh15_control.h>
#include <dexh15/dexh15_define.h>

#include <iostream>
#include <thread>

/**
 * @brief 执行预定义动作
 * @details 步骤:
 *          (1)~(7) 同位置模式前置流程
 *          (8) 执行预定义动作 1 至 10
 *          (9) 下使能电机
 *          (10) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate)) { return 1; }
    if (!dexh15->initModbusDevice(device_address)) { dexh15->disconnectModbus();
return 1; }
    if (!dexh15->initMotorPosition(device_address)) { dexh15->disconnectModbus();
return 1; }
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手连接状态为" << (is_connected ? "在线" : "离线") << std::endl;

    if (!dexh15->setMotorControlMode(device_address,
                                     paxini::bot::dexh15::POSITION_CONTROL_MODE)) {
        dexh15->disconnectModbus();
        return 1;
    }
    if (!dexh15->enableMotor(device_address)) { dexh15->disconnectModbus(); return
1; }

    paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};

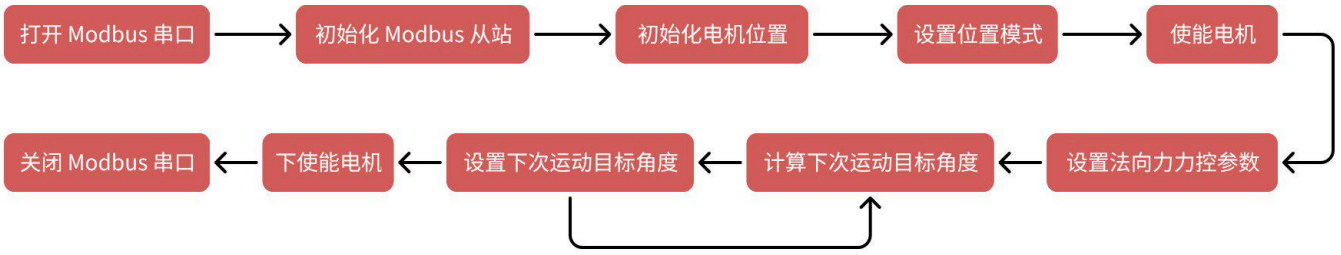
    // 执行内置动作，内置动作共有 10 个
    for (int i = 1; i <= 10; i++) {
        if (!dexh15->runAction(device_address, i, false)) {
```

```
            std::cerr << "执行内置动作失败，编号为 " << i << std::endl;
        }
        std::cout << "执行内置动作成功，编号为 " << i << std::endl;
        std::this_thread::sleep_for(std::chrono::seconds(1));
        // 电机位置回零
        dexh15->setMotorTargetPosition(device_address, zero_positions);
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    bool is_success = dexh15->disableMotor(device_address);
    std::cout << "电机下使能" << (is_success ? "成功" : "失败") << std::endl;
    is_success = dexh15->disconnectModbus();
    std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_success ? " 成功" :
" 失败") << std::endl;
    return 0;
}
```

3.2.5 力位混合控制模式

DexHandSDK 法向力位混合控制模式的使用步骤:



示例代码:

```
#include <dexh15/dexh15_control.h>

#include <iostream>
#include <chrono>
#include <thread>

/**
 * @brief 法向力力位混合控制模式
 * @details 步骤:
 *          (1)~(7) 同位置模式前置流程
 *          (8) 设置法向力力控参数
 *          (9) 计算下次运动的目标角度
 *          (10) 设置下次运动目标角度
 *          (11) 重复步骤 (9) 和 (10)，直至握住物体且法向力恒定
 *          (12) 下使能电机
 *          (13) 关闭 Modbus 串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
```

```
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate)) { return 1; }
    if (!dexh15->initModbusDevice(device_address)) { dexh15->disconnectModbus();
return 1; }
    if (!dexh15->initMotorPosition(device_address)) { dexh15->disconnectModbus();
return 1; }
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "灵巧手连接状态为" << (is_connected ? "在线" : "离线") << std::endl;

    if (!dexh15->setMotorControlMode(device_address,
                                     paxini::bot::dexh15::POSITION_CONTROL_MODE)) {
        dexh15->disconnectModbus();
        return 1;
    }
    if (!dexh15->enableMotor(device_address)) { dexh15->disconnectModbus(); return
1; }

    // 法向力力控参数
    paxini::bot::dexh15::ForceCtlParam force_ctl_param{
        {50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50},
        {5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5},
        {true, true, true, true, true, true}};

    // 初始位姿
    std::vector<double> original_angle = {0, 0, 0, 0, 0, 0, 0, 0, 500, 0, 0};

    // 设置法向力力控参数
    if (!dexh15->setCommonForceCtlParam(device_address, original_angle,
force_ctl_param)) {
        std::cerr << "设置法向力力控参数失败" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }

    // 执行法向力位混合模式控制
    auto start_time = std::chrono::steady_clock::now();
    auto duration = std::chrono::seconds(30);
    while (std::chrono::steady_clock::now() - start_time < duration) {
        std::vector<double> command;
        if (!dexh15->computeCommonForceTargetCommand(device_address, command)) {
            std::cerr << "计算下次运动的目标角度失败" << std::endl;
            break;
        }
        if (!dexh15->setJointPositionsAngle(device_address, command)) {
            std::cerr << "设置下次运动目标角度失败" << std::endl;
            break;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
    }
```

```
    // 电机回到零位
    paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};
    dexh15->setMotorTargetPosition(device_address, zero_positions);

    bool is_success = dexh15->disableMotor(device_address);
    std::cout << "电机下使能" << (is_success ? "成功" : "失败") << std::endl;
    bool result = dexh15->disconnectModbus();
    std::cout << "断开 Modbus 串口设备 " << hand_port_num << (is_connected ? " 成功"
: " 失败") << std::endl;
    return 0;
}
```

3.2.6 相机功能

DexHandSDK 相机功能的使用步骤:



示例代码:

```
#include <dexh15/dexh15_camera.h>

#include <iostream>
#include <opencv2/opencv.hpp>

/**
 * @brief 相机功能
 * @details 步骤:
 *          (1) 打开相机串口
 *          (2) 设置分辨率和帧率
 *          (3) 获取一帧图像
 *          (4) 显示图像
 *          (5) 释放相机串口
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Camera> dexh15_camera =
        std::make_shared<paxini::bot::dexh15::DexH15Camera>();
    std::string camera_port_num = "/dev/video4";
    try {
        // 打开相机串口
        dexh15_camera->connectCameraDevice(camera_port_num);
        // 设置分辨率和帧率
        if (!dexh15_camera->setCameraConfig(640, 480, 30)) {
            std::cout << "设置相机参数失败" << std::endl;
        }
        std::cout << "设置相机参数成功" << std::endl;
        // 获取一帧图像
        cv::Mat frame = dexh15_camera->getFrame();
        // 显示图像
```

```
            dexh15_camera->showImage("img", frame);
            // 释放相机串口
            dexh15_camera->releaseCameraDevice();
            std::cout << "释放相机串口" << std::endl;
        }
        catch (...) {
        }
        return 0;
    }
```

---

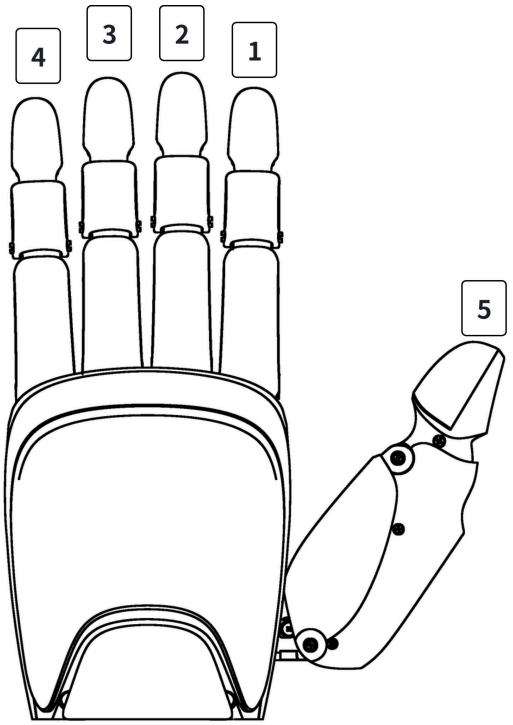
4. 核心概念

本节介绍使用 SDK 前需要了解的关键概念，理解这些概念有助于您更高效地使用各 API。

4.1 层级寻址与资源映射

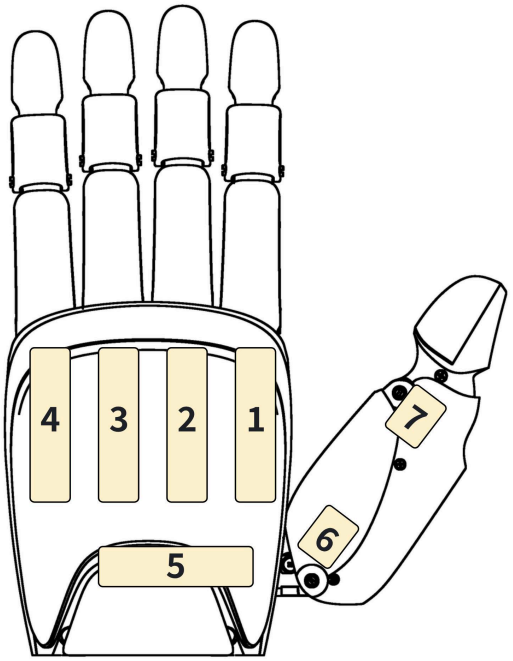
4.1.1 手指编号

GMH15 灵巧手共有 5 根手指，其手指编号方式如下所示。



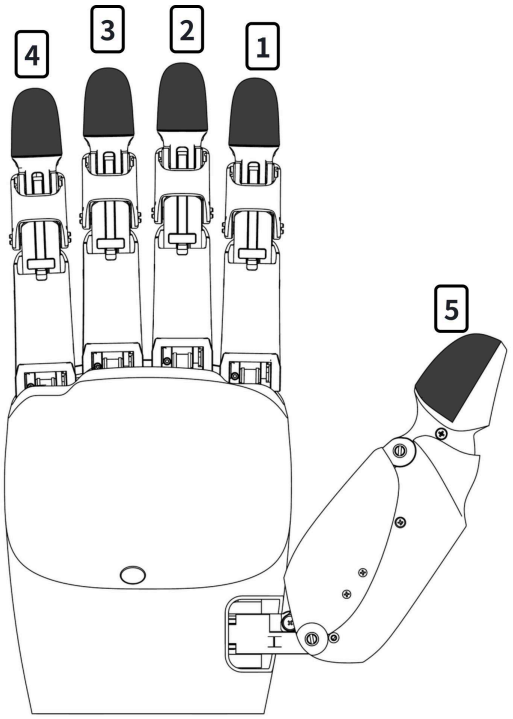
4.1.2 电机编号

GMH15 灵巧手共有 7 个电机，其电机编号方式如下所示。



4.1.3 传感器编号

GMH15 灵巧手共有 5 个触觉传感器模组，其传感器编号方式如下所示。



4.2 `slave_id` — 设备寻址

`slave_id` 是 SDK 中所有直接控制类 API 的第一个参数，用于在共享总线上寻址特定设备。

通讯协议	`slave_id` 含义	典型值
Modbus-RTU	从站 ID（0x01 ~ 0xF7）	1、2、5
USB 直连	固定为 0（点对点，无需寻址）	0

4.2.1 数值归一化

SDK 使用**归一化相对值**代替硬件绝对值，屏蔽不同批次硬件的参数差异。

维度	含义	归一化范围
电机位置	0 与 1000 表示电机的 2 处行程极限位	0 – 1000
关节位置（屈曲）	0 = 完全张开，1000 = 完全握紧	0 – 1000
关节位置（侧摆）	-500 与 500 表示关节侧摆的左右 2 处极限角度	-500 – 500
最大电机电流	0 = 零电流，1000 = 允许最大电流	0 – 1000
最大电机速度	0 = 零速度，1000 = 允许最大速度	0 – 1000
电机速度	0 = 静止，1000 = 最大速度	0 – 1000
电机电流	0 = 无力，1000 = 最大电流	0 – 1000

4.2.2 控制模式

灵巧手支持三种底层电机控制模式，适用于不同场景：

模式	枚举值	适用场景
位置模式	`POSITION`	精确姿态控制（最常用）
速度模式	`VELOCITY`	需要持续匀速运动的场景
电流模式	`CURRENT`	直接力矩控制、底层力控研究

> ⚠️ **注意**：切换控制模式前建议先令手回到零位（`resetToZeroPosition`），以避免模式切换瞬间产生非预期运动。

---

5. 常见问题

5.1 连接问题

5.1.1 Modbus 手动连接失败

原因：波特率错误，或从站地址错误

解决方案：用 Modbus 自动连接方式，来连接 GMH15

```
/**
 * @brief 自动连接当前串口下的第一个灵巧手设备，适用于单设备场景
 * @param hand_port_num: 灵巧手串口编号
```

```
 * @param device_address: 返回连接上的设备地址
 * @return 成功返回 true，失败返回 false，程序错误抛出异常
 */
bool connectModbusAuto(const std::string& hand_port_num, int16_t& device_address);
```

5.2 控制模式设置问题

5.2.1 设置控制模式失败

原因：在电机使能状态下，设置控制模式

解决方案：用以下方法下使能电机，再设置控制模式

```
/**
 * @brief 控制电机下使能
 * @param slave_address: 从站地址
 * @return 电机下使能成功返回 true，失败返回 false。
 */
bool disableMotor(int16_t slave_address);
```

5.3 电机控制问题

5.3.1 设置电机位置失败

原因：灵巧手当前状态为不可运行状态、电机未使能或控制模式不为位置模式

解决方案：

- **灵巧手当前状态为不可运行状态**

  用节 3.2.2 故障诊断进行初步排查，若存在电机断联错误码，请与我司联系（https://www.paxini.com/），进行维修

- **电机未使能**

  用以下方法使能电机，再设置电机位置

  ```
  /**
   * @brief 控制电机上使能
   * @param slave_address: 从站地址
   * @return 使能电机成功返回 true，失败返回 false
   */
  bool enableMotor(int16_t slave_address);
  ```

- **控制模式不为位置模式**

用以下方法将控制模式设置为位置模式，再设置电机位置

```
/**
 * @brief 定义电机控制模式，默认是位置模式。
 * @param mode: 电机的控制模式，可以是电流模式、速度模式、位置模式
 * @return 设置成功返回 true，失败返回 false。
 */
bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
```

5.3.2 设置电机目标电流失败

原因：电机目标电流大于电机允许的最大电流、电机未使能或控制模式不为位置模式

解决方案：

- **电机未使能**

  用以下方法使能电机，再设置电机目标电流

  ```
  /**
   * @brief 控制电机上使能
   * @param slave_address: 从站地址
   * @return 使能电机成功返回 true，失败返回 false
   */
  bool enableMotor(int16_t slave_address);
  ```

- **控制模式不为力矩模式**

  用以下方法将控制模式设置为力矩模式，再设置电机目标电流

  ```
  /**
   * @brief 定义电机控制模式，默认是位置模式。
   * @param mode: 电机的控制模式，可以是电流模式、速度模式、位置模式
   * @return 设置成功返回 true，失败返回 false。
   */
  bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
  ```

- **电机目标电流大于电机允许的最大电流**

  用以下方法查询电机允许的最大电流

  ```
  /**
   * @brief 获取从站设备所有电机的最大电流配置
   * @param slave_address: 从站地址
   * @return 所有电机的最大电流配置
  ```

```
 */
Dex15MotorCurrent getMotorMaxCurrentConfig(int16_t slave_address);
```

若电机允许的最大电流小于电机目标电流，用以下方法修改电机最大电流

```
/**
 * @brief 定义电机最大电流
 * @param max_current: 范围 [0, 1000]
 * @return 设置成功返回 true，失败返回 false。
 */
bool setMotorMaxCurrent(int16_t slave_address, const Dex15MotorCurrent&
max_current);
```

5.4 传感器数据问题

5.4.1 无按压时，传感器合力不为 0

原因：未校准传感器

解决方案：在读取传感器数据前，用以下方法进行校准传感器

```
/**
 * @brief 标定灵巧手触觉传感器
 * @param slave_address: 从站地址
 * @return 标定成功返回 true，失败返回 false
 */
bool calibrateSensor(int16_t slave_address);
```

5.5 图像处理问题

5.5.1 无法识别相机串口号

原因：type-c 线损坏

解决方案：检查 type-c 线是否完好。若 type-c 线损坏，请更换 type-c 线

5.5.2 获取到的图像黑屏

原因：相机接线松动或损坏

解决方案：请与我司联系（https://www.paxini.com/），进行维修

---

6. 注意事项

1. 在使用 SDK 之前，请确保已经正确安装了 SDK 环境，并了解了灵巧手产品的基本操作和使用方法。
2. 在使用 SDK 时，请按照函数接口的说明进行调用，避免出现参数错误或调用顺序错误等问题。
3. 在控制灵巧手运动时，请根据实际情况设置合理的运动参数，避免出现运动失控或碰撞等问题。
4. 在获取传感器数据时，请根据实际情况设置合理的数据采集频率，避免出现数据丢失或延迟等问题。
5. 在使用 SDK 时，如果遇到问题或错误，请参考错误代码和错误信息进行排查和解决。

---

7. 版权声明

1. 本文档会定期进行检查和修正，更新后的内容将出现在新版本中。本文档中的内容或信息如有变更，恕不另行通知。
2. 使用产品前请阅读本文档。
3. 请保管好本文档，以便随时阅读与参考。
4. 本文档所记载的内容，不排除有错误或遗漏的可能性，如对本文档中的内容有疑问，请与我司联系（https://www.paxini.com/）。